AWS CDK Introduction

Explore how to use AWS CDK to define and deploy cloud infrastructure programmatically. Learn to initialize projects, use constructs and stacks, and manage deployments with CDK CLI. This lesson prepares you to implement infrastructure as code efficiently and aligns with AWS Certified Developer exam objectives.

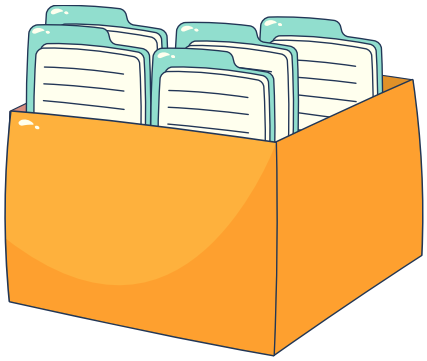
The AWS Certified Developer – Associate (DVA-C02) exam includes several tasks related to deploying applications, automating infrastructure, and using tools like the AWS Cloud Development Kit (CDK). As cloud development increasingly involves managing infrastructure as code (IaC), the ability to define and deploy AWS resources programmatically is essential.

This lesson aligns closely with exam objectives in the “Deployment” domain, specifically where the exam expects candidates to implement and deploy infrastructure as code using tools such as AWS CDK. Mastery of the concepts here will help us pass the exam and prepare us for real-world scenarios where rapid, repeatable, and secure infrastructure deployment is critical.

What is AWS Cloud Development Kit (CDK)?

The **AWS Cloud Development Kit (CDK)** is an open-source framework for defining cloud infrastructure using general-purpose programming languages. Unlike traditional *infrastructure as code (IaC) solutions* that use declarative templates like JSON or YAML, the CDK allows developers to use TypeScript. Python, Java, or C# to define infrastructure resources. The CDK then syntl this code into AWS CloudFormation templates used to provision and manage resources.

Ask



Why should we use AWS CDK?

Now that we have understood the basics of the CDK, let's examine why we may want to use the CDK:

- **Developer productivity:** The CDK enables developers to dynamically define resources using loops, conditionals, and variables in code. This reduces repetitive declarations and makes complex deployments easier to manage. For example, we can create ten S3 buckets in a loop or configure a set of Lambda functions with similar parameters from one consistent pattern.
- **Modularity:** CDK supports breaking infrastructure into reusable modules called constructs. This allows teams to encapsulate patterns (like a Lambda connected to an API Gateway) and reuse them across multiple projects. It promotes consistency and accelerates onboarding for new developers who can use existing building blocks.
- **Maintainability:** Keeping infrastructure code in the same repository as our application code simplifies life cycle management. It makes changes traceable via version control, allows integration with CI/CD workflows, and ensures our deployment logic evolves alongside application requirements. This alignment is especially useful when deploying microservices or serverless functions.
- **Best practices:** By default, many CDK constructs apply AWS security and scalability best practices. For instance, an `s3.Bucket` created via CDK will have private access and encryption enabled unless explicitly changed. These

Ask

defaults help enforce compliance and reduce configuration errors, making our infrastructure more secure and reliable.

Supported languages and CDK toolkit overview

The CDK supports TypeScript, JavaScript, Python, Java, C#, and Go (experimental). Most developer teams prefer TypeScript or Python for their active community and comprehensive documentation.

To work with CDK, install the AWS CDK Toolkit, a CLI that provides commands to manage the life cycle of CDK apps:

main.sh Shell-20

```
npm install -g aws-cdk
```

Installing AWS CDK

This installs the CDK CLI globally on our machine, enabling commands like `cdk init`, `cdk synth`, and `cdk deploy`.

Ask

Here is a list of key CLI commands:

- `cdk init`: This initializes a new CDK project with boilerplate files for the selected programming language.
- `cdk synth`: This converts our CDK code into a CloudFormation template.
- `cdk diff`: This compares the current deployed stack with our local changes to preview what will change.
- `cdk deploy`: This deploys our app to AWS using CloudFormation.
- `cdk destroy`: This removes all resources in the stack safely through CloudFormation.

Setting up the AWS CDK

Before we can begin writing infrastructure with the CDK, our development environment needs to be properly configured. This includes installing essential tools and authenticating with AWS. Follow these foundational steps to prepare for building CDK applications using Python:

1. **Install Node.js (version 16.x or higher):** Node.js is required because the CDK CLI is distributed through npm (the Node.js package manager).
2. **Install the AWS CDK CLI (as discussed above):** The CLI is our primary tool for managing CDK applications. Installing it globally allows access to CDK commands from any terminal session.
3. **Install Python (for Python CDK apps):** Python is one of the supported languages in CDK. We'll also need to install `pip` and set up a virtual environment when working in Python.
4. **Configure our AWS credentials using `aws configure`:** This command stores our AWS access key, secret key, region, and output format so CDK can authenticate and deploy resources to our account.

After setup, initialize a new CDK app. We can initialize a Python app using command given below:

Ask

main.sh Shell-20

```
cdk init app --language python
```

The details are presented as follows:

- `cdk init`: This begins a new project.
- `app`: This indicates the type of CDK project.
- `--language python`: This specifies Python as the project language.

Similarly, we can initialize an app for TypeScript using the command given below:

main.sh Shell-20

```
cdk init app --language typescript
```

Ask

Initializing an app

In this case, the different parts mean the following:

- `cdk init`**:** This starts a new CDK project.
- `--language typescript`**:** This specifies TypeScript as the language.

CDK components

Let's discuss the main building blocks that developers use to define their cloud infrastructure.

- **App:** In the AWS CDK, an app serves as the root of the infrastructure definition. It acts as the container for one or more stacks, and is typically the entry point of our CDK project. Think of it as the overall application blueprint. For example, in a real-world multi-environment setup, a single CDK app may contain a `DevStack`, `TestStack`, and `ProdStack`—each representing the same application but with environment-specific configurations.
- **Stack:** A stack in CDK corresponds to a single AWS CloudFormation stack. It defines a unit of deployment and contains the resources that should be created, updated, or deleted together. Each stack is independently deployable and manageable. For example, in a microservices architecture, each service could be deployed via its own stack, allowing teams to manage them independently without risk to unrelated components.
- **Construct:** Constructs are the basic building blocks in CDK and represent one or more AWS resources. They can be as simple as a single S3 bucket or as complex as an entire VPC setup with subnets and NAT gateways. Constructs support encapsulation and reuse, making them ideal for modeling higher-level components in a modular and standardized way. In practi team might develop a custom `ApiGatewayWithLambda` construct that combines common configuration and use it across multiple stacks to enforce consistency.

Ask

- **L1 (low-level) constructs:** L1 constructs map directly to AWS CloudFormation resources. These are thin wrappers over raw resource definitions and offer complete control over configuration. They're useful when working with new or rarely used AWS features not yet supported by higher-level constructs. For instance, `CfnBucket` gives us access to every parameter available in a native CloudFormation S3 bucket resource.
- **L2 (high-level) constructs:** L2 constructs simplify AWS resource creation by abstracting common defaults, validation, and secure practices. These constructs are developer-friendly and help enforce best practices with minimal effort. For example, `s3.Bucket` allows us to enable versioning and encryption with just a few parameters, making it ideal for most real-world use cases.
- **L3 (patterns) constructs:** L3 constructs bundle multiple services into opinionated design patterns for common workloads. These are powerful tools for teams that want to standardize architecture. For instance, `aws-apigateway.LambdaRestApi` creates an entire serverless API integration with one construct, which is great for bootstrapping new applications or aligning to architecture blueprints.

Ask

Writing the first CDK app in Python or TypeScript

Once initialized, our project will have an entry point file and a stack definition. Here's how to create a simple S3 bucket.

The code snippet below shows how we can create a simple S3 bucket in Python and Typescript, using AWS CDK:

Pyth...

```
from aws_cdk import core as cdk
from aws_cdk import aws_s3 as s3

class MyFirstStack(cdk.Stack):
    def __init__(self, scope: cdk.Construct, id: str, **kwargs):
        super().__init__(scope, id, **kwargs)
        s3.Bucket(self, "MyFirstBucket")
```

S3 bucket creation using AWS CDK in Python and Typescript

Working with the AWS Construct Library

The **AWS Construct Library** provides packages for nearly every AWS servi allowing developers to quickly configure services like S3, Lambda, Dynam and VPC. For example, instead of writing long YAML or JSON definitions for an

Ask

S3 bucket with versioning and encryption enabled, we can do it in a single line of CDK code.

CDK also makes it easy to wire together services. We can create a Lambda function and give it access to read from an S3 bucket or respond to events, all within a few lines of code.

This integration ensures we maintain security best practices and deploy tightly-integrated cloud applications more efficiently.

Synthesis and deployment using CDK CLI

Once our CDK app is defined, the CDK CLI handles the key actions for deployment:

- `cdk synth`**:** This generates the equivalent AWS CloudFormation template from our CDK code. This is useful for reviewing what will be deployed before executing changes.
- `cdk deploy`**:** This pushes our application stack to AWS. CDK will provision or update resources based on the current state of our code and deployed infrastructure.
- `cdk destroy`**:** This safely tears down all resources associated with a stack, ensuring clean cleanup. It prompts for confirmation before proceeding.

Best practices for CDK projects

Here is a list of best practices for managing CDK projects:

- **Structure projects modularly:** Splitting large stacks into smaller, focused constructs improves clarity, reusability, and maintainability. For example, instead of having all resources in one file, we might isolate networking components into one construct, database resources into another, and so on. This structure helps teams work independently and simplifies debugging.

Ask

- **Use version control:** Storing our CDK code in a Git repository makes collaboration more efficient. Version control enables pull requests, code reviews, and rollback capabilities. It also supports best practices in continuous integration and deployment pipelines.
- **Test and validate:** Testing CDK applications improves confidence before deployment. Use `cdk diff` to preview infrastructure changes and ensure alignment with expectations. For unit testing, CDK provides libraries such as `@aws-cdk/assertions` (TypeScript) and `assertions.Template` (Python) to inspect synthesized templates and assert resource properties. These allow for validating construct behavior programmatically.
- **Parameterize environments:** Managing differences between development, testing, and production environments is crucial. Use CDK context variables or conditionals in our code to adjust configuration settings. This way, the same codebase can support multiple deployment targets with minimal duplication.
- **Cleaning up and destroying stacks safely:** Proper clean up avoids orphaned resources and unexpected charges. Always use `cdk destroy` to remove a deployed stack. This ensures that CloudFormation handles the resource teardown correctly and avoids manual deletion mistakes that can lead to inconsistencies. For critical environments, enable termination protection to prevent accidental deletion.

Conclusion

In this lesson, we explored how the AWS Cloud Development Kit (CDK) empowers developers to define cloud infrastructure using programming languages like Python and TypeScript. We learned to initialize and structure CDK projects, use constructs and stacks to model resources, and leverage the CDK CLI to synthesize and deploy applications to AWS.

Ask

By adopting CDK, we benefit from abstraction, code reuse, built-in best practices, and integration with standard development workflows. These capabilities not only boost developer productivity but also promote infrastructure consistency across environments.

Ask